

Web Infra - Formative 1

Exploring and Testing APIs with Postman

1. API Choice and Base URL

API: GitHub REST API, Gists endpoint

Base URL: <https://api.github.com>

I picked the Gists endpoint on GitHub's REST API because it lets me run through the full CRUD cycle (Create, Read, Update, Delete) on a single resource type, and the authentication side of it is real and enforced. Unlike some of the free demo APIs out there, GitHub genuinely rejects a request if the token is missing or wrong, so this actually tests something. It was also a lot faster to set up than a full OAuth2 flow like Reddit's, while still counting as proper API-key-style authentication for this assignment.

2. Authentication Method

Method: Personal Access Token (PAT), sent as an Authorization: Bearer {{auth_token}} header on every request.

GitHub doesn't work like APIs that hand you a /login endpoint and a short-lived session token. Instead, you generate a long-lived Personal Access Token once, manually, through GitHub Settings > Developer settings > Personal access tokens, and reuse it across every request from then on. I scoped mine to gist only, since that's the only permission this assignment actually needed.

To confirm the token actually worked before doing anything else, I sent a GET /user request. A 200 OK with my own account details in the response body was enough to confirm it was valid.

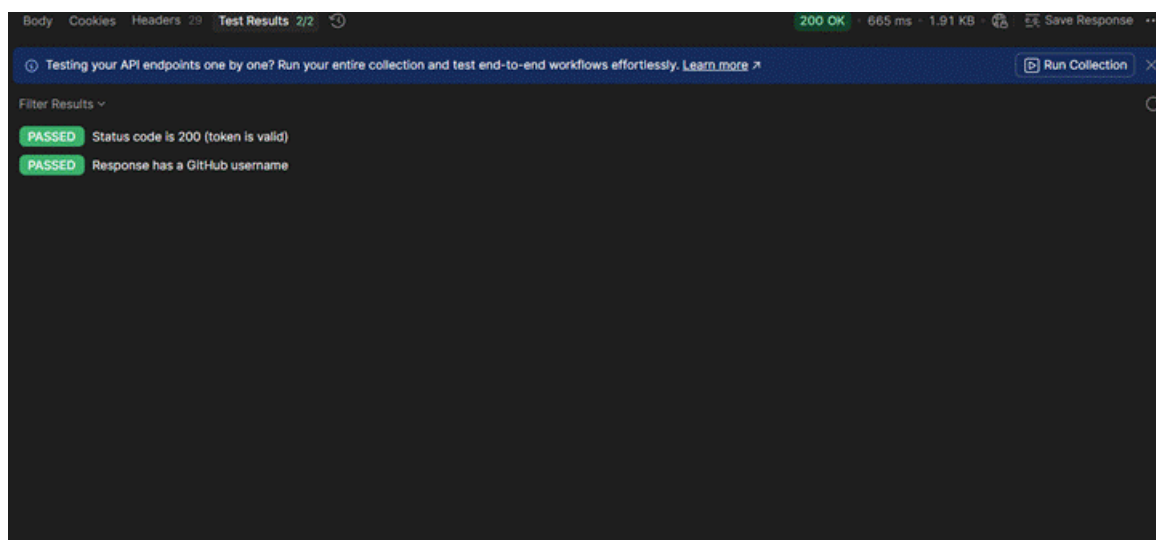


Figure 1: “0 - Verify Authentication” response, 200 OK, 2/2 tests passed.

3. CRUD Operations

Resource: Gists

Operation	Method	Endpoint	Result
Create	POST	/gists	201 Created
Read (all)	GET	/gists	200 OK
Read (one)	GET	/gists/{id}	200 OK
Update	PATCH	/gists/{id}	200 OK
Delete	DELETE	/gists/{id}	204 No Content

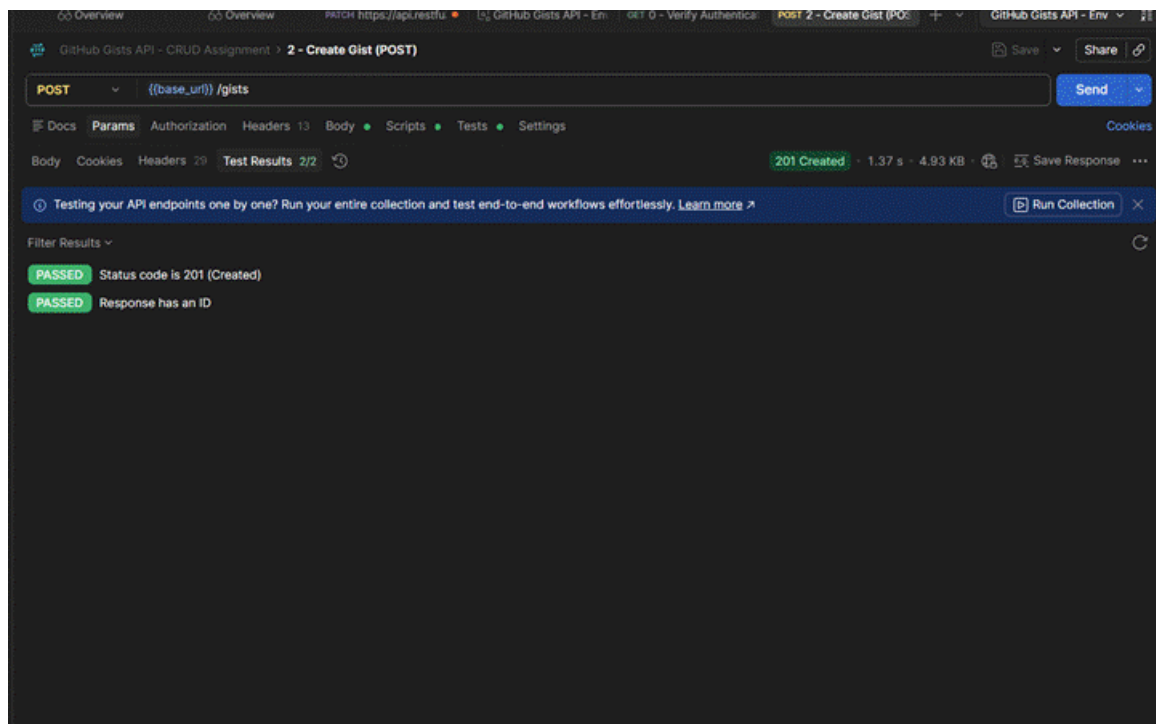


Figure 2: Create Gist (POST), 201 Created, 2/2 tests passed.

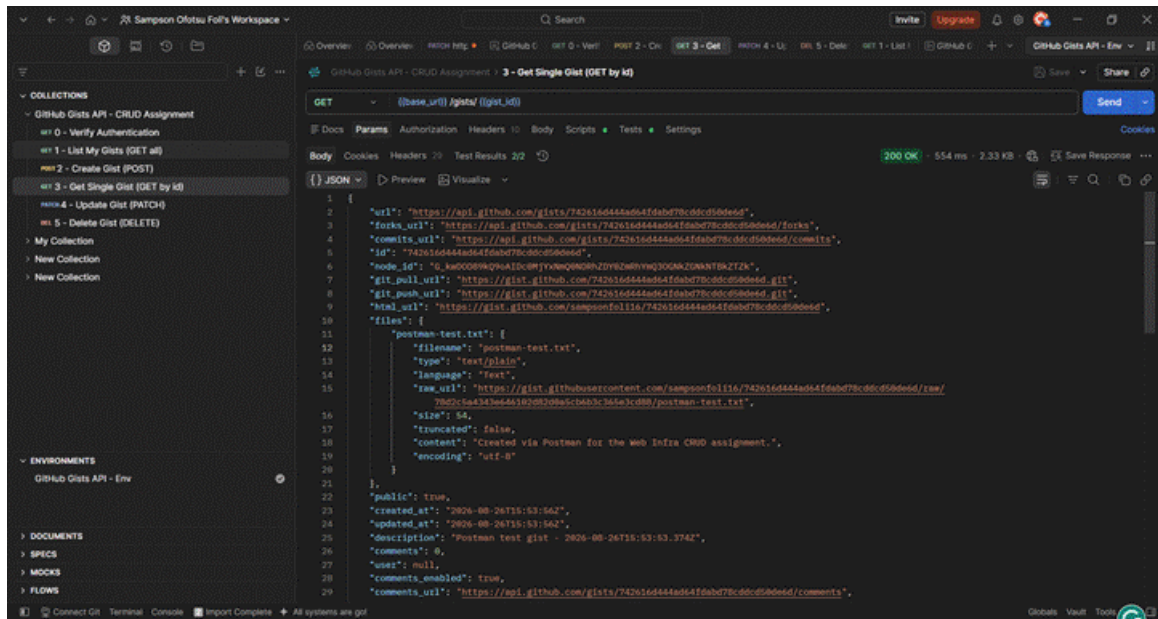


Figure 3: Get Single Gist (GET by id), 200 OK, response body showing the created gist's data.

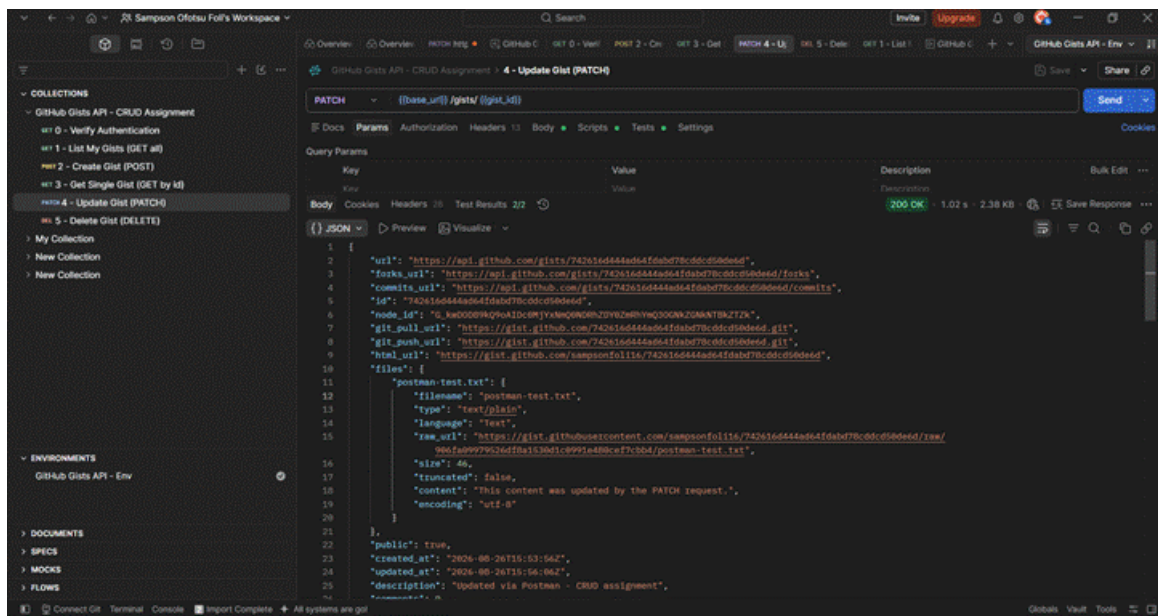


Figure 4: Update Gist (PATCH), 200 OK, response body showing the updated description and content.

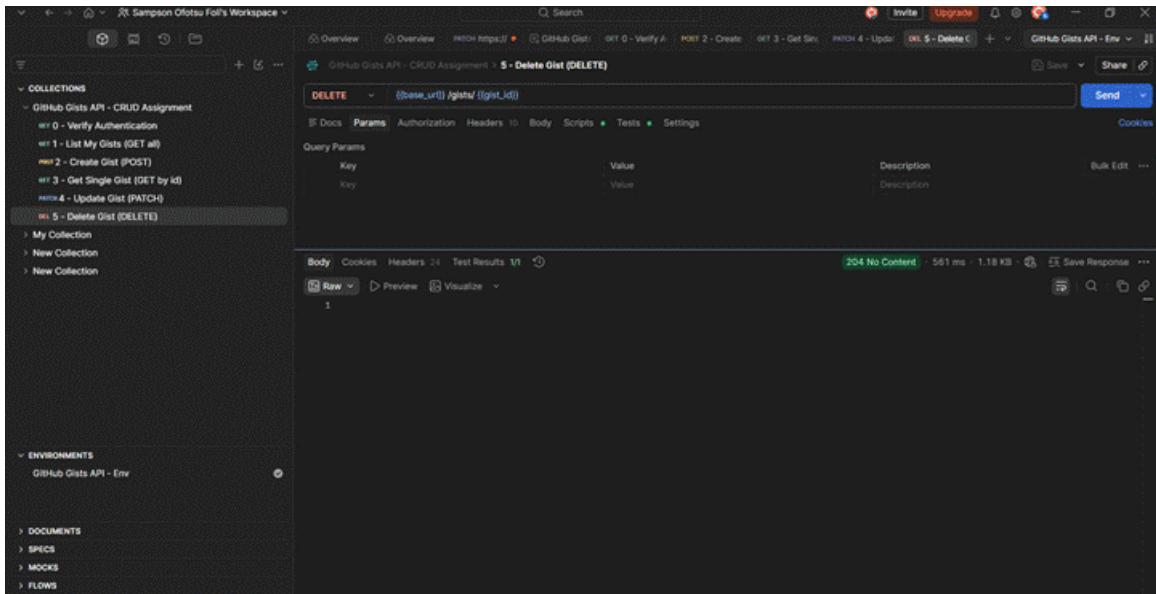


Figure 5: Delete Gist (DELETE), 204 No Content, 1/1 test passed.

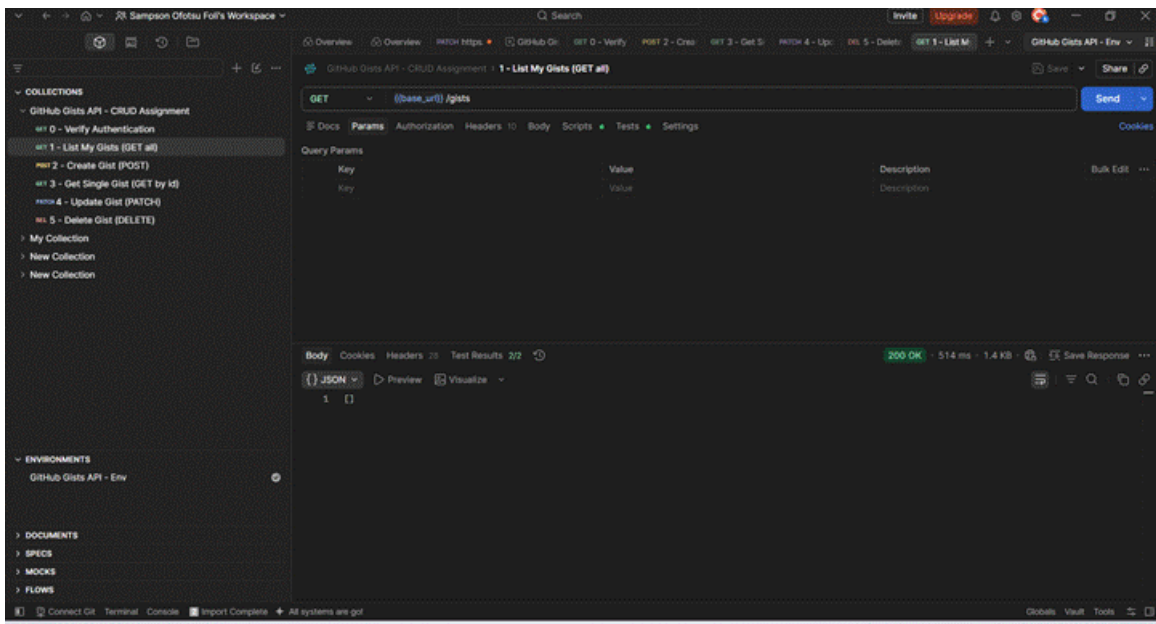


Figure 6: List My Gists after deletion, empty array, confirming the delete genuinely removed the resource.

4. Postman Environment & Variables

I set up an environment called “GitHub Gists API - Env” to hold:

- `base_url`, `https://api.github.com`
- `auth_token`, the PAT, marked as a secret/masked variable
- `gist_id`, the id of whichever gist is currently active
- `gist_description`, a description generated fresh on every run

Every request pulls these in with `{{variable}}` syntax instead of hardcoded values. That means the whole collection could be reused against a different account just by swapping in a different environment, nothing in the requests themselves would need to change.

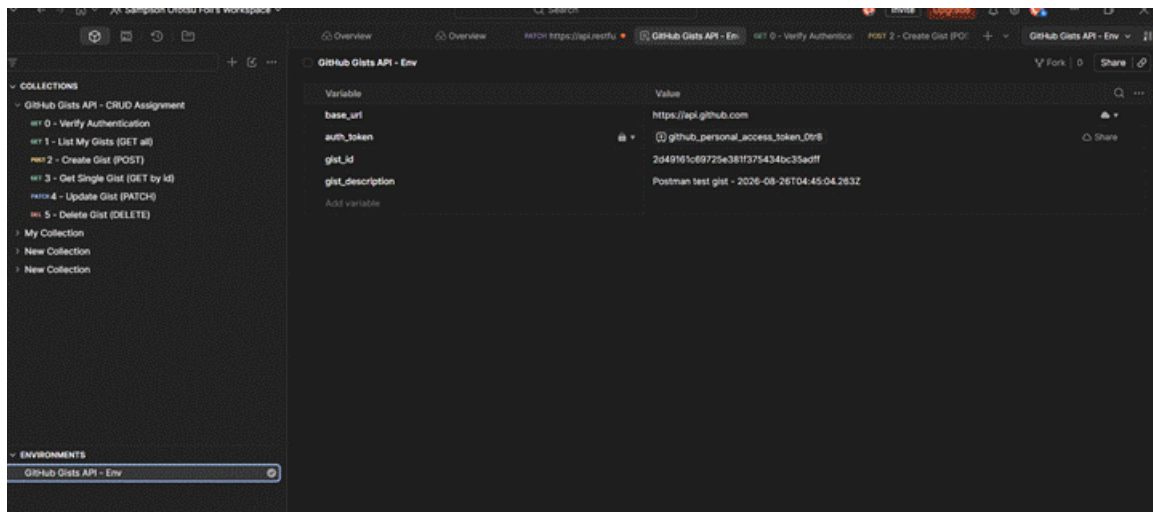


Figure 7: Environment showing gist_id and gist_description auto-filled after the Create request ran.

5. Automation

I wrote a few scripts so the collection isn't just a set of static, hardcoded requests:

- **Test script on Create:** once POST /gists succeeds, this pulls the new gist's id straight out of the response and saves it into the environment with `pm.environment.set("gist_id", ...)`. Every request after that (Get, Update, Delete) picks up that saved id automatically, so I never have to copy an id between requests by hand.
- **Pre-request script on Create:** builds a fresh, timestamped description each time the request runs, so running the collection twice in a row doesn't leave a pile of identical-looking gists behind.
- **Collection-level pre-request script:** runs before every request and checks that `auth_token` actually has a value, printing a warning to the console if it's empty.

6. Collection Runner

I ran the full collection, all 6 requests in order, through Postman's Collection Runner.

Result: 11/11 tests passed, 0 errors, 0 skipped.

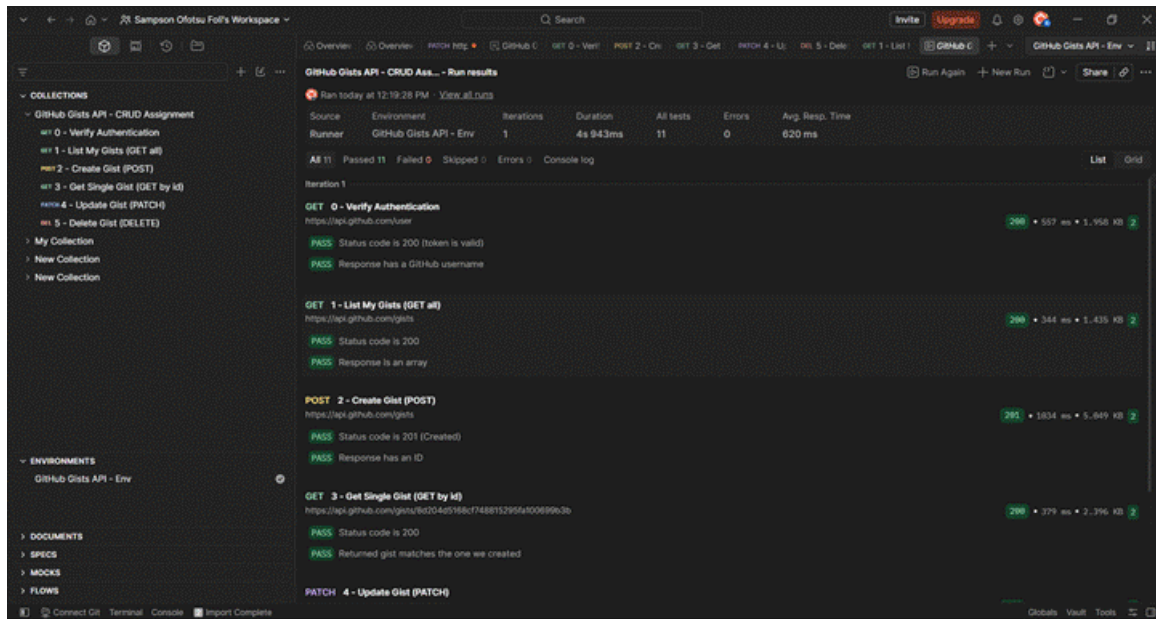


Figure 8: Collection Runner summary, 11/11 tests passed, 0 errors.

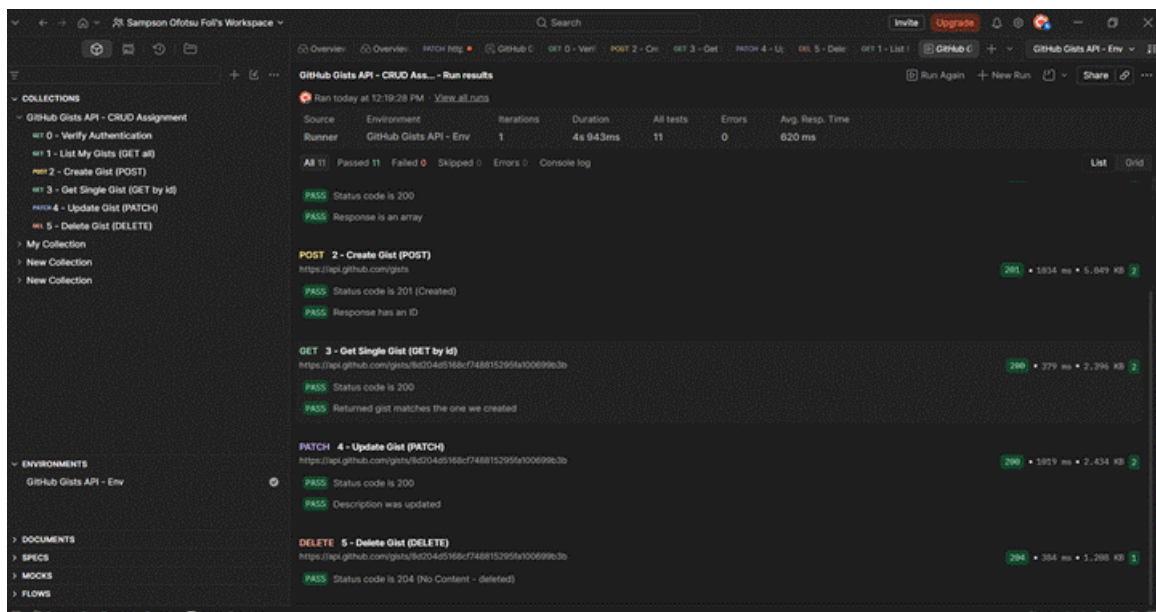


Figure 9: Remainder of the run, showing Update and Delete results.

7. Repository Contents

- **github-gists-api.postman_collection.json**: the exported collection
- **github-gists-api.postman_environment.json**: the exported environment, token value left blank so each user provides their own
- **REPORT.docx**: this document
- **screenshots/**: all screenshots referenced above